INDEX

S.No	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
01.	1/7	Introduction & Basic HTML	CO_1	10	
02	8/7	Design HTML using audio	CO_1	10	
03.	15/7	Design HTML using video	CO_1	9	
04.	22/7	program using JS (arrays)	CO_1	10	
05.	29/7	Develop dynamic webpage	CO_1	10	
06.	5/8	Implement modules in node.js.	CO_2	10	
07.	12/8	MPA application using express & node.js.	CO_1	10	
08.	19/8	CodeTantra (static web page)	CO_1	10	
09.	2/9	CodeTantra (portfolio)	CO_1	10	
10.	9/9	CodeTantra (Simple Calculator)	CO_1	10	
11.	16/9				
12.	23/9				

Scanned with OKEN Scanner

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech - ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222. Website: www.anits.edu.in email

Department of Computer Science & Engineering (AI & ML)

Academic Year:	R. Jateshkumar	Semester:	
Name of the Student:	[redacted]	Roll Number:	A24126552164
Name of the Lab:	FS Lab	Timings:	

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	1	4	4	5	2	15		
8/7	2	4	4	5	2	15		
15/7	3	4	3	5	2	14		
22/7	4	4	4	5	2	15		
29/7	5	4	3	5	2	14		
5/8	6	4	4	5	2	15		
12/8	7	4	4	5	1	14		
19/8	8	4	4	5	1	14		
2/9	9	4	4	4	1	13		
9/9	10.	4	4	4	2	14		

Name of the Faculty: Ch. Sruthi

Faculty Signature: Sruthi

Scanned with OKEN Scanner

* Introduction to HTML and CSS:

HTML (Hypertext Markup Language) is the standard markup language used to create and structure content on webpages. It defines the basic structure of webpage using elements and tags such as headings paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interepted by web Browsers to display the content in a structured and meaningful manner. HTML provides structure and semantic organization of a webpage.

CSS (cascading style sheets) is a style sheet language used to control the presentation & visual appearance of HTML Documents. It is used to define properties such as colors, fonts, background, spacing borders, alignment, dimensions and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sized and devices CSS can be applied using inline, internal or external, HTML and CSS work together to develop webpages, where HTML provides the structure and content, while CSS provides styling. The combination of both, forms the foundation

Scanned with OKEN Scanner

of modern web development and enables the creation of organized, attractive and user-friendly web interfaces.

* Difference between MERN STACK and FULL stack:

MERN stack is a java script based technology stack used for developing modern web applications. MERN stack stands for mongoDB, Express.js, React.js, Node.js, mongoDB is used for data base management, Express.js handles the back end framework and API's, React.js is used to develop frontend user Interface and node.js provides server side run-time environment.

→ Full-Stack Development refers to the development of both frontend and backend of a web application along with database integration. A Full stack developer works with technologies.

*HTML Elements:

1. HTML tags - Define the structure and elements of a web page.
2. Headings - used to display titles and headings using <h1> to <h6>
3. Paragraphs - used to display text using <p> tag
4. Links - used to display navigate between webpages using <a> tags.
5. Images - used to display images using <image> tag.
6. Lists - used to create ordered & unordered lists using <ol>, <ul>, <li>.
7. Tables - used to organize data in rows and columns
8. Forms - used to collect user input using elements such as text boxes, buttons and check boxes.
9. Div and Span - used to group and organize web page content.
10. Semantic Elements - Elements such as <header>, <nav>, <selection>, <article>, <footer> provide meaningful webpage structure.

CSS Components:

1. selectors - Select HTML elements to apply styles.
2. properties - Define the visual characteristics of elements.
3. values - Specify the setting of a css property.
4. Colors - Define text, background, borders, colors.
5. Fonts - Control font family, size, style and weight.
6. Background - set background colors and images.
7. Borders - Define the border style, width and radius.

Scanned with OKEN Scanner

8. Margin - Controls space outside an element.

9. padding - Controls the space inside an element.

10. Layout - Controls arrangement of elements using flexbox, grid and positioning.

* HTML vs HTML5:-

HTML	HTML5
older version of HTML	Latest major version of HTML.
Limited multimedia support	Supports audio and video
Uses external plugins for multimedia.	No external plugs needed.
Has fewer semantic tags.	provides semantic tags.
No <canvas> element.	Support <canvas>.
Limited webstorage	Supports local storage

* CSS vs CSS3:

CSS	CSS 3
Earlier version of css.	Advanced version of css
Basic styling features	provides advanced styling features.
Limited animations.	Supports animations and transitions.
No flexible layout systems	Supports flexbox and Grid

* Design a static webpage using HTML and CSS

-> pomodoro Timer - using HTML, CSS

```
<!Doctype html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
      body {
        font-family: Arial;
        background: #f5f5f5;
        text-align: center;
        padding: 50px;
      }
      timer {
        width: 300px;
        margin: auto;
        padding: 30px;
        back-ground: white;
        border-radius: 15px;
        box-shadow: 0 0 10px grey;
      }
      h1 {
        color: #1C74C3C;
      }
```

Scanned with OKEN Scanner

```
time of
    font-size : 50px;
    margin : 20px;
}
button of
    padding: 10px 20px;
    margin : 5px;
    border : none;
    border-radius : 5px;
    background : #e74c3c;
    Color : white;
}
</style>
</head>
<body>
    <div class = "timer">
    <h1> Pomodoro Timer </h1>
    <div class = "time"> 25:00 </div>
    < button> start </button>
    < button> Reset </button>
  </div>
</body>
</html>
```



Scanned with OKEN Scanner

1. Design an HTML code using audio tags :-

Code :

```
<!DOCtype html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> my favorite song </h2>
    <audio controls>
        <source src="song.mp3" type="audio/
                                    mpeg">
        your browser doesnot support audio tag.
    </audio>
</body>
</html>
```

O/P : Displays an audio player with play, pause, and volume controls.

Scanned with OKEN Scanner

2. Design a HTML code using video tags.

Code:

```
<!Doctype html>
<html>
<head>
   <title> Video example </title>
</head>
<body>
    <h2> my Video </h2>
    <video width = "400" controls>
        <source src="video.mp4" type="video/mp4">
    </video>
</body>
</html>
```

O/p: Displays a video player with play, pause, volume, and full screen controls.

3. Write a program using Javascript (Arrays, Functions).

Code:

```
let numbers = [12,45,8,23,56,19,34,67,90,11];
Function display(arr){
    Console.log("Array Elements:");
    for(let i=0; i<arr.length; i++){
        Console.log(arr[i]);
    }
}
Function findsum(arr){
    let sum=0;
    for(let i=0; i<arr.length; i++){
        sum+=arr[i];
    }
    return sum;
}
```

Experiment-04

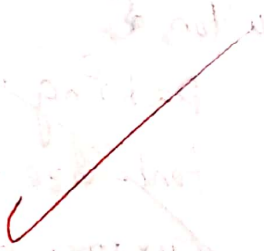
Develop Dynamic webpage using JavaScript.

Program:

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic Webpage </title>
    <style>
        body {
            text-align: center;
            font-family : Arial;
            margin_top : 100px;
        }
        button {
            padding : 10px 20px;
            font-size : 16px;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <h1 id="heading"> Welcome to My Webpage
    </h1>
    <p id = "message"> Click the button to change
        this webpage </p>
    <button onclick, "ChangePage()"> ClickMe
    </button>
```

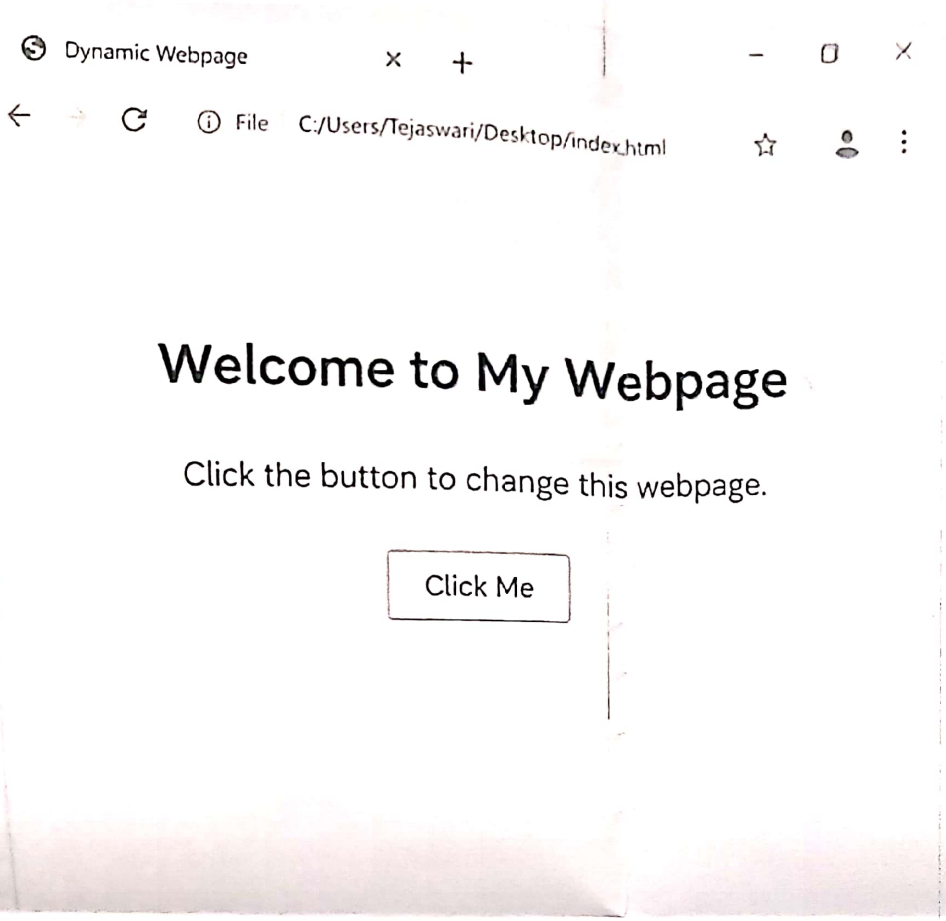
Scanned with OKEN Scanner

```
<script>
    function changepage(){
    document.getElementById("heading").
        innerHTML = "Welcome to javascript!";
    document.getElementById("message").
        innerHTML = "The Webpage has been
                Changed Successfully";
    document.body.style.backgroundColor
            = "lightblue";
            }
</script>
</body>
</html>
```

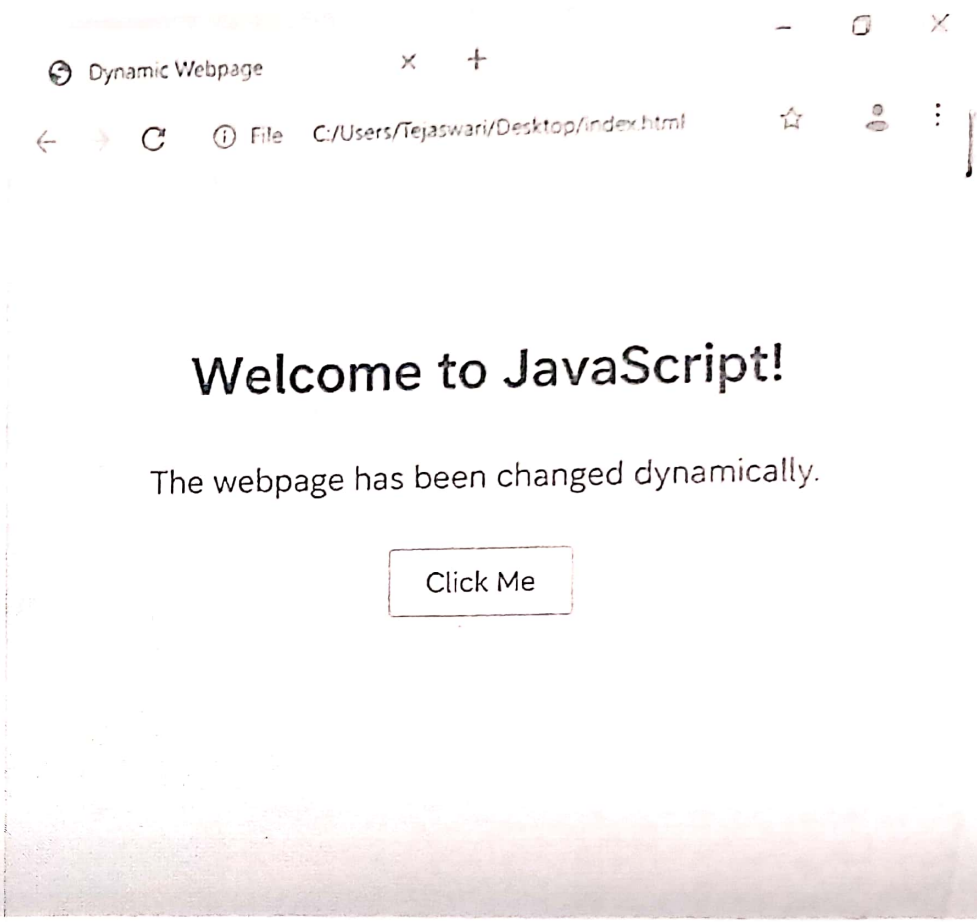


Scanned with OKEN Scanner

Output:



Before clicking the button



After clicking the button

Scanned with OKEN Scanner

Experiment-05

5. Implement modules in Node.Js.

In Node.js, you can implement modules using two distinct systems: CommonJS (CJS), which is traditional default, and ESmodules (ESM), the modern java script standard.

1. The CommonJS Method (default):

CommonJS uses module.exports to share code and require() to import it. This system loads modules synchronously.

a. Create the module (math.js):

```
Const add = (a,b) => a+b;
Const subtract = (a,b) => a-b;
module.exports = {
    add,
    subtract
};
```

b. use the module (app.js):

```
Const math = require ('./math');
Console.log (math.add (10,5));
Console.log (math.subtract (10,5));
```

Output:

15
5

2. The ES modules Method (modern standard):

ES modules use export to share code and import to use it. To make Node.js recognize ES modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a. Create the module (calculator.js)

```
export Const multiply = (a,b) => a+b
export const divide = (a,b) => a/b
Const calculatorName = "SuperCalc";
export default CalculatorName;
```

b. Use the module (main.js)

```
import calculatorName, {multiply, divide}
                from './calculator.js';
Console.log (CalculatorName);
Console.log (multiply (4,2));
Console.log (divide (4,2));
```

Output:

SuperCalc
8
2.

3. Using Built-in core modules:

Node.js also comes equipped with dozens of built-in core modules (like fs for file systems or path for file routing) that don't need installation. We will load them by module names:

Scanned with OKEN Scanner

- // CommonJS style

```
Const fs = require ('fs');
```

- // ES module style

```
import fs from 'fs';
```

Here are some of the most Commonly used ones;

1. fs = file System operations.
2. http - HTTP Server and Client.
3. path - file path utilities.
4. os - operating system utilities.
5. events - Event handling.
6. util - utility functions.
7. stream - Stream handling.
8. crypto - Cryptographic functions.
9. url - URL parsing.
10. query string - URL query string handling.

Example: Simple HTTP Server.

```
http.CreateServer ((req, res) => {
    res.writeHead (200, { 'Content-Type':
                          'text/html' });
    res.end ('Hello World!');
}).Listen (8080);
```

Output: Hello World!

Scanned with OKEN Scanner

Experiment - 06.

6. Develop Mini application using express and Node.js

- Simple mini application using node JS + Express JS.

Student Management System:

1. Add a Student
2. View all students
3. Delete a student
4. Users Express.js routes
5. Simple HTML interface.

Project structure:

```
student-app/
  ├ server.js
  └ public
      └ index.html
```

server.js:

```
const express = require("express");
const app = express();
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));
app.get("/students", (req, res) => {
    res.json(students);
});
//Add
app.post("/students", (req, res) => {
    const student = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course,
    };
    students.push(student);
    res.json({ message: "student added
                succesfully",
        student: student });
});
//Delete
app.delete("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    students = students.filter(student => student.id !
                ==id);
    res.json({
        message: "student deleted succesfully" });
});
// start server
app.listen(PORT, () => {
    console.log(`Server running at https://local host:
                ${PORT}`);
});
```

Scanned with OKEN Scanner

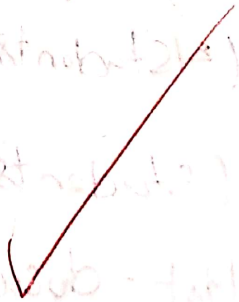
public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title> Student Management </title>
</head>
<body>
  <h1> Student Management System </h1>
  <input type="text" id="name"
   placeholder="Enter Student Name">
  <input type="text" id="course"
   placeholder="Enter course">
  <button onclick="addStudent()"> Add Student
                                  </button>
  <h2> Student List </h2>
  <ul id="studentList"></ul>
  <script>
    function addStudent() {
      let name = document.getElementById("name").
                                    value;
      let course = document.getElementById("course").
                                    value;
      fetch("/students", {
          method: "POST",
          headers: {
            "Content-Type": "application/json"},
          body: JSON.stringify({
              name: name,
              course: course
          })
      })
      .then(response => response.json())
      .then(data => {
          alert(data.message);
document.getElementById("name").value = "";
document.getElementById("course").value = "";
      displayStudents();
      });
    }
function displayStudents(){
  fetch("/students").then(response=>response.json())
  .then(students => {
    let list = document.getElementById("studentList");
    list.innerHTML = "";
students.forEach(student => {let li =
  document.createElement("li");
      li.innerHTML = student.id + "-" +
                     student.name + "-" +
                     student.course + "<button
onclick='deleteStudent(" + student.id + ")'> Delete
</button>";
```

Scanned with OKEN Scanner

```
list.appendChild(li);
            });
        });
    }
    function deleteStudent(id) {
        fetch("/students/" + id, { method: "DELETE
        .then(response => response.json())
        .then(data => {
            alert(data.message);
            displayStudents();
        });
    }
    displayStudents();
</script>
</body>
</html>
```

Output:



Scanned with OKEN Scanner

1. Terminal Output (after running `node server.js`)

2. Initial Output in Browser (no students)

3. After adding a student (alert message)

4. Student list after adding one student

5. After adding another student (alert message)

6. After deleting a student (alert message and updated list)

Scanned with OKEN Scanner

Experiment - 07

07. Create a Functional Component and class Component using react.

Aim: To develop a react application using functional components and class components to display student details, manage state, handle button clicks, and dynamically update data on a web page.

Functional Component:

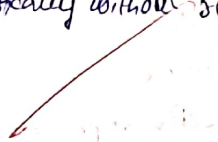
A Functional component is a JavaScript function that returns JSX to display the user interface. It uses react hooks such as useState to manage state and update the webpage dynamically.

Program:

```
import React, {useState} from
'react';
function studentFunctional(){
    const[name, setName] = useState("Jeetesh");
    Const [Course, setCourse] = useState("ncsm");
    Const [Count, setCount] = useState(0);
    Const changeName = () => {
            setName("Jeetesh");
        };
    Const changeCourse = () => {
        setCourse("CSD");
    };
    Const IncrementCount = () => {
        setCount(Count + 1);
    };
return(
  <div style = {{
      border = "2px solid blue",
      padding : "20px",
      Margin : "20px",
      borderRadio : "10px",
      textAlign : "center"
  }}>
  <h1> Student Details </h1>
  <h2> Functional Component </h2>
  <p> <b> Name: </b> {name} </p>
  <p> <b> Course: </b> {Course} </p>
  <p> <b> Button clicks : </b> {Count} </p>
  <button onClick = {ChangeName}>
        ChangeName </button>
  <button onClick = {ChangeCourse} Change
        Course </button>
  <button onClick = {IncrementCount}> Increment
        </button>
  </div>
);
```

Scanned with OKEN Scanner

Output: When the user clicks the buttons, the name, Course and count change dynamically without reloading the webpage.



. Class Component:

A class component is a ES6 JavaScript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState().

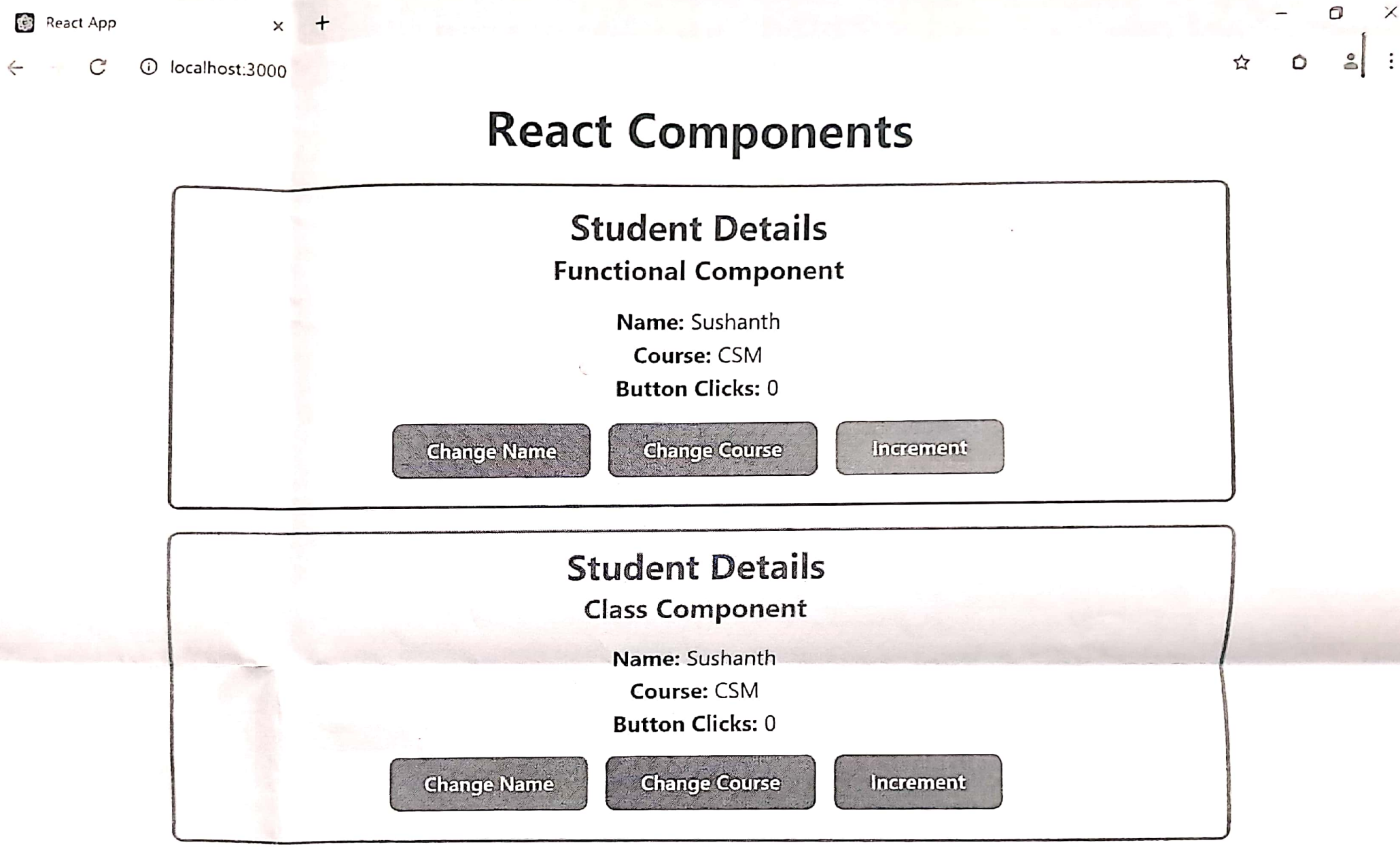
Program:

```
import React, {Component} from
'react';
class studentclass extends Component{
    constructor(props){
        super(props);
        this.state ={
            name: "Jeetesh",
            course: "Computer Science",
            count: 0
        };
    }
    ChangeName = () => {
        this.setState({
            name: "Jeetesh"
        });
    };
    changeCourse = () => {
        this.setState({
            course: "Com" });
    };
```

Scanned with OKEN Scanner

```
IncrementCount=() =>{
    this.setState((prevState) => ({
        Count: prevState.Count+1
    }));
};
render(){
  return(
    <div style={{
        border: "2px solid green";
        padding: "20px";
        margin: "20px",
        borderRadius: "10px",
        textAlign: "center"
    }}>
    <h1> Student Details </h1>
    <h2> class Component </h2>
  <p><b>Name: </b> {this.state.name} </p>
  <p><b> Course: </b> {this.state.course} </p>
  <p><b> Button clicks : <b> {this.state.count} </p>
<button onClick = {this.changeName}>
        ChangeName </button>
<button onClick = {this.changeCourse}>
        Change Course </button>
<button onClick = {this.IncrementCount}> IncrementCount
        </button> </div>
```

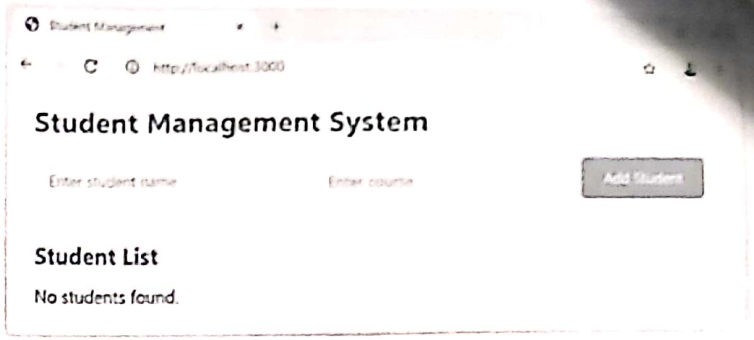
Scanned with OKEN Scanner



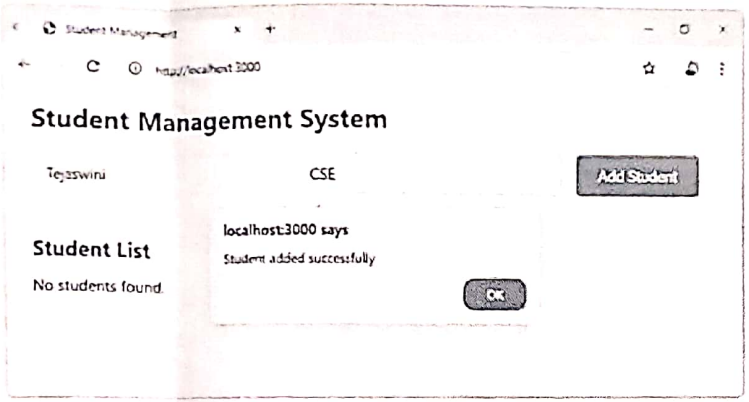
Scanned with OKEN Scanner



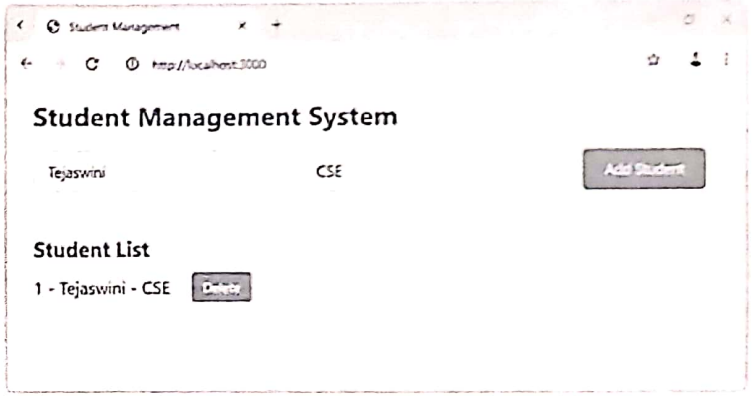
1. Terminal Output (after running `node server.js`)



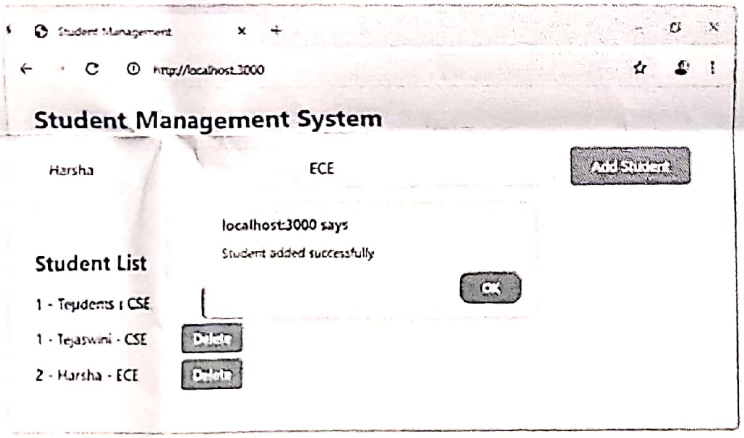
2. Initial Output in Browser (no students)



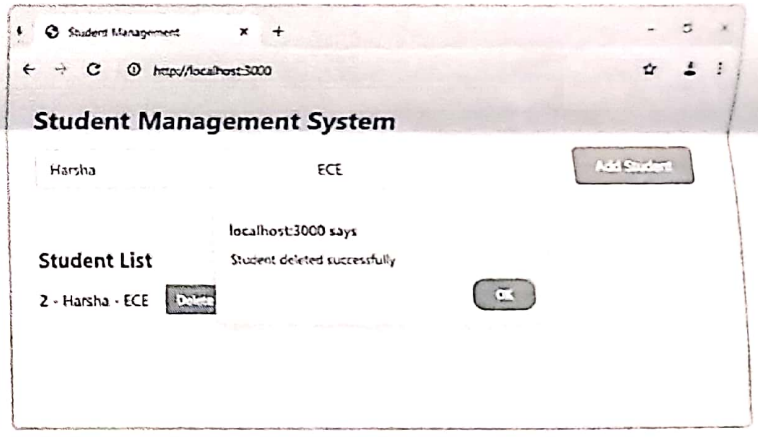
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Scanned with OKEN Scanner

Experiment-08

8. Implement HTTP methods using JSON server.

Aim: To implement HTTP methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (Create, Read, Update, Delete) operations on student records.

Theory: JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate between a client and a server.

- GET - Retrieves data from the server.
- POST - Adds new data to the server.
- PUT - Replaces an existing record.
- PATCH - Updates selected fields of a record.
- DELETE - Removes a record from the server

a. Create a JSON server:

Step1: Initialize the project.

Open the vscode terminal and execute the following commands:

```
mkdir JSONServer
cd JSON server
npm init -y
npm install json server
```

Step2: Create db.json

Create a file named db.json and add following student data.

```
{ "students": [
  { "id": "1",
    "name": "Jeetesh",
    "Course": "Csm",
    "marks": '90' },
  { "id": "2"
    "name": "[illegible]",
    "Course": "Csm",
    "marks": 90
  },
  { "id": "3",
    "name": "Himesh",
    "Course": "IT",
    "marks": 88
  }
```

Step3: Start the JSON server

For the current JSON server version, execute:

```
npx json-server db.json
```

Scanned with OKEN Scanner

The server start at:

http://localhost:3000

The student records can be accessed using:

http://localhost:3000/students

2. Implement of HTTP methods:

a. GET method: The GET method is used to retrieve student records from JSON server.

Code:

```
fetch('http://localhost:3000/students')
  .then(response => response.json())
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  });
```

Output

b. POST method: The post method is used to insert a new student record into server.

code:

```
fetch('http://localhost:3000/students', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'Rahul',
        Course: 'CSE',
        marks: 92
    })
})
.then(response => response.json())
.then(data => {
    console.log(data);
})
```

Scanned with OKEN Scanner

```
.catch(error => {
    console.log(error);
});
```

Output:

C. PUT METHOD: The put method is used to replace an existing student record with updated data.

Code:

```
fetch('http://localhost:3000/students/1', {
  method: 'PUT',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    name: 'Jeetesh',
    Course: 'AIML',
    marks: 95
  })
})
.then(response => response.json())
.then(data => {
    console.log(data); })
```

```
.catch(error => {
    console.log(error);
});
```

output:

D. PATCH METHOD: The patch method is used to update only selected fields of an existing student record.

code:

```
fetch('http://localhost:3000/students/2', {
  method: 'PATCH',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    marks: 96
  })
})
.then(response => response.json())
.then(data => {
console.log(data); })
.catch(error => {
```

Scanned with OKEN Scanner

```
    console.log(error);
});
```

Output:

E DELETE METHOD: The Delete method is used to remove an existing student record from the server

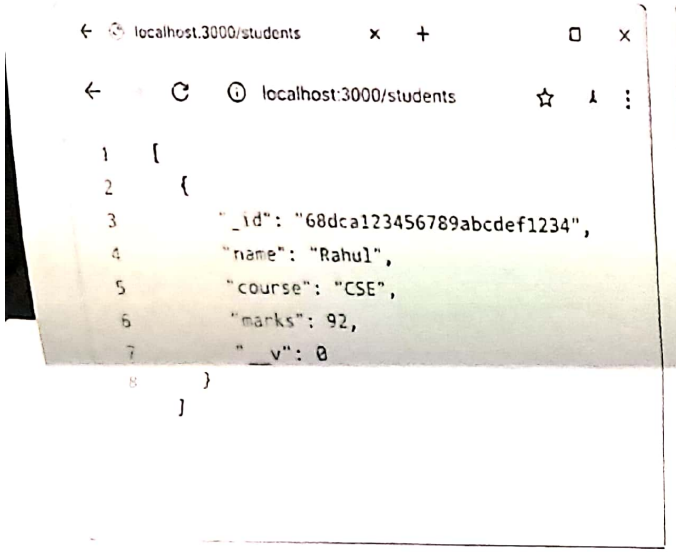
Code:

```
fetch('http://localhost:3000/students/3', {
    method: 'DELETE' })
.then(response => {
    if(response.ok){
        console.log('Student deleted successfully');
    }
})
.catch(error => {
    Console.log(error);
});
```

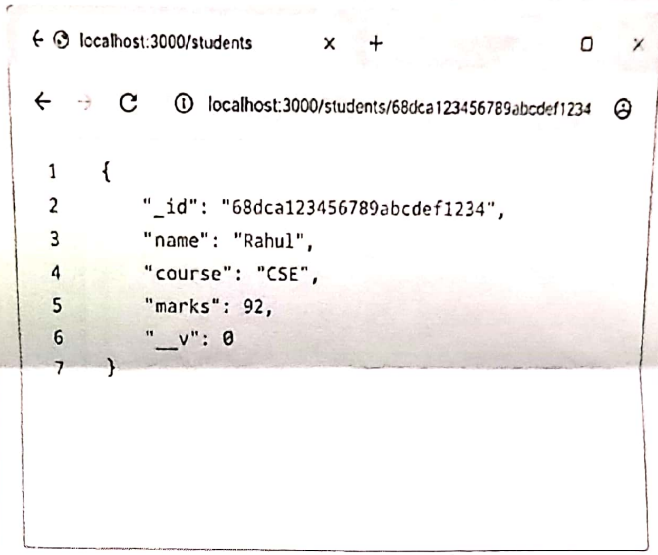
Output: ~~catch(err~~

Scanned with OKEN Scanner

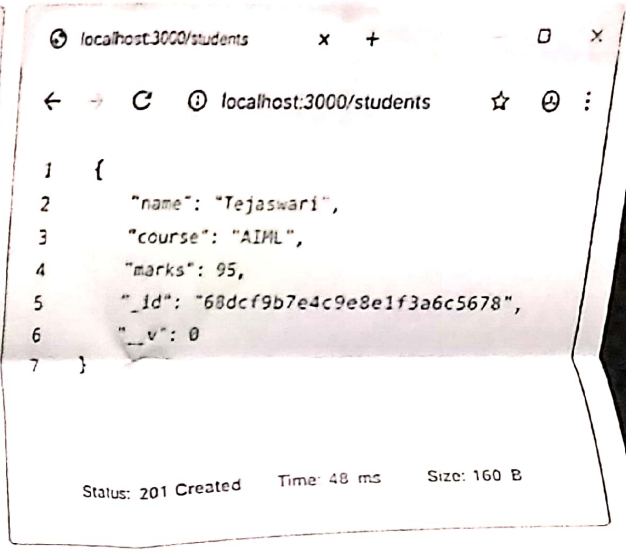
1. GET All Students (http://localhost:3000/students)



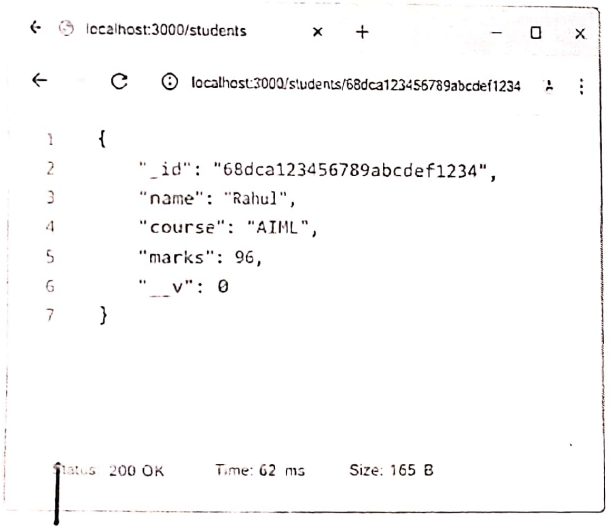
2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)



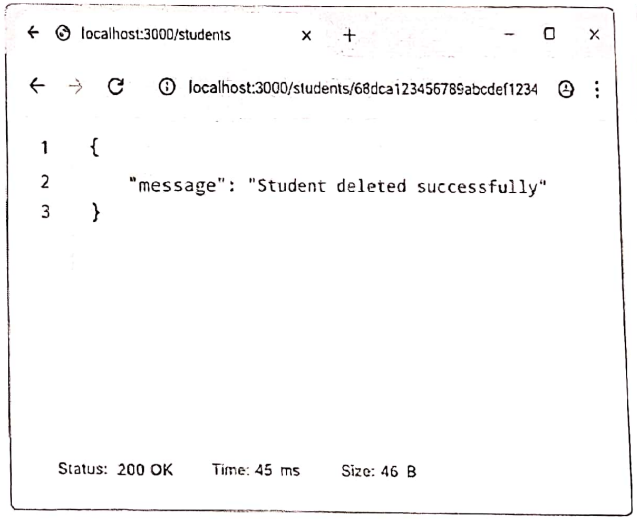
3. POST (Create Student) – Using Postman



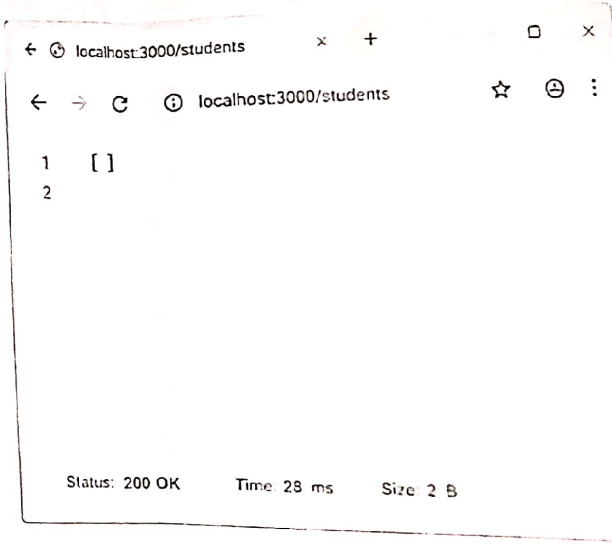
4. PUT (Update Student) – Using Postman



5. DELETE (Remove Student) – Using Postman



6. GET All Students (After Deletion)



Scanned with OKEN Scanner

· After executing the POST, PUT, PATCH, and DELETE operations in sequence, the remaining student will be:
output

Thus, HTTP methods GET, POST, PUT, PATCH, and DELETE were successfully implemented using JSON Server. The CRUD operations were performed on student records, the data was retrieved, inserted, updated and deleted using HTTP requests.

Scanned with OKEN Scanner

Experiment-09

09. Develop CRUD operations using MongoDB:-

Aim: To develop a web application, using Node.js, Express.js and MongoDB to perform CRUD operations (create, read, update, delete) on student records.

Theory: MongoDB is a NoSQL database that stores data in flexible, JSON-like documents, CRUD operations are used to manage data in a database.

Mongoose is an Object Data modeling (ODM) library for mongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

1. Installation and project setup:

```
Mkdir mongoDB-CRUD
Cd MongoDB-CRUD
npm init -y
npm install express mongoose
```

Program: Server.js

```
const express = require('express');
Const mongoose = require ('mongoose');
Const app = express();
Const PORT = 3000;
app use (express.json());
mongoose.connect ('mongobd://127.0.0.1:27017/College")
.then(() => console.log ("mongo DB Connected "))
.catch (err => console.log (err));
// Create Student
Const student schema = new
mongoose.schema ({
    name: {
      type: string,
      required : true
    }
    Course: {
      type: String,
      required: true
    },
    marks: {
      type : Number
      required : true
    }
});
// Create student model
Const student:
    mongoose.model ("student", Student schema);
// Create Add
app.post ('/students', asyn c (req, res) => {
    try {
```

```
const student = new Student (req.body);
const savedStudent = await student.save();
res.status(201).json(savedStudent);
} catch(error){
  res.status(400).json({message:
      error.message});
  }
});
// READ: Get all students
app.get('/students', async (req,res) => {
  try{
    const students = await
    Student.find();
    res.status(500).json({message: error.
        message});
  }
});
// READ: Get student by ID
app.get('/students/:id', async(req,res) => {
  try{
    const student = await
    Student.findById(req.params.id);
    if(!student){
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.status(200).json(student);
  } catch(error){
    res.status(400).json({message:
        error.message});
  }
});
// update
app.put('/students/:id', async(req,res) => {
  try{
    const student = await
    Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      {new: true, runValidators: true}
    );
    if(!student){
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.status(200).json(student);
  } catch(error){
    res.status(400).json({message:
        error.message});
  }
});
```

Scanned with OKEN Scanner

// DELETE

```
app.delete('/students/:id', async (req, res) => {
    try {
        const student = await Student.
            findByIdAndDelete(
            req.params.id);
        if(!student){
            return res.status(404).json({
                message: 'student not found'
            });
        }
        res.status(200).json({
            message: "student deleted
                successfully"
        });
    } catch (error){
        res.status(400).json({message:
            error.message});
    }
});
```

Outputs:

Scanned with OKEN Scanner

1. Terminal Output

```
PS C:\MongoDB_CRUD> node server.js
MongoDB connected
Server running on port 3000
```

2. Chrome Browser (Home Page)

localhost:3000

MongoDB CRUD API

Server is running...

3. Create (POST) – Add Student

POST http://localhost:3000/students Send

Params Authorization Headers (9) Body Scripts Settings

raw JSON

```
{
  "name": "Tejaswari",
  "course": "CSE",
  "marks": 85
}
```

Body Cookies Headers (7) Test Results

201 Created 155 ms 265 B

```
[
  {
    "_id": "68da1f2e4c9e8e1f3a6c1234",
    "name": "Tejaswari",
    "course": "CSE",
    "marks": 85,
    "__v": 0
  }
]
```

4. Read (GET) – Get All Students

GET http://localhost:3000/students Send

Params Authorization Headers (7) Body Scripts Settings

raw JSON 200 OK 32 ms 512 B

```
[
  {
    "_id": "68da1f24c9e8e1f3a6c1234",
    "name": "Tejaswari",
    "course": "CSE",
    "marks": 85,
    "__v": 0
  },
  {
    "_id": "68da1f3a4c9e8e1f3a6c5678",
    "name": "Rohit",
    "course": "AIML",
    "marks": 92,
    "__v": 0
  }
]
```

5. Read (GET) – Get Student by ID

GET http://localhost:3000/students/68da1f2e4c9e8e1f3a6c1234 Send

Params Authorization Headers (7) Body Scripts Settings

raw JSON 200 OK 28 ms 245 B

```
{
  "_id": "68da1f2e4c9e8e1f3a6c1234",
  "name": "Tejaswari",
  "course": "CSE",
  "marks": 85,
  "__v": 0
}
```

6. Update (PUT) – Update Student

PUT http://localhost:3000/students/68da1f2e4c9e8e1f3a6c1234 Send

Params Authorization Headers (9) Body Scripts Settings

raw JSON Body Cookies Headers (7) Test Results

200 OK 24 ms 268 B

```
{
  "_id": "68da1f2e4c9e8e1f3a6c1234",
  "name": "Tejaswari",
  "course": "AIML",
  "marks": 95,
  "__v": 0
}
```

7. Delete (DELETE) – Remove Student

DELETE http://localhost:3000/students/68da1f3a4c9e8e1f3a6c5678 Send

Params Authorization Headers (9) Body Scripts Settings

Body Doc Sketch Tidbits Cell

200 OK 20 ms 210 B

```
{
  "message": "Student deleted successfully"
}
```

8. MongoDB Compass – Database View

MongoDB Compass

Local

localhost:27017

college

students

admin

config

local

ADD DATA

Open MongoSH Refresh

college.students

Documents Aggregations Schema Indexes

Filter { }

_id	name	course	marks	...
68da1f2e4c9e8e1f3a6c1234	Tejaswari	AIML	95	0
68da1f3a4c9e8e1f3a6c5678	Rohit	AIML	92	0

Scanned with OKEN Scanner